



ALGORYTMY I STRUKTURY DANYCH

Termin lab.: piątek 13:30

Kierunek: Informatyka

Grupa lab.: 5

SPRAWOZDANIE Z LABORATORIUM

3. Algorytmy grafowe

Autor:

PRO100KatYT

Prowadzący zajęcia:

mgr inż. Domomod

Rok akademicki 2025/2026

Spis treści

1	O programie	1
1.1	Opis	1
1.2	Prezentacja działania programu	1
2	Wizualizacja wygenerowanych grafów	2
3	Wykresy benchmarków	3
4	Podsumowanie	4

1 O programie

1.1 Opis

Program służy do generowania spójnych, skierowanych grafów acyklicznych oraz wczytywania grafów podanych przez użytkownika w postaci listy następników. Głównym celem programu jest reprezentacja grafów na trzy różne sposoby (macierz sąsiedztwa, lista następników i lista krawędzi) oraz wykonanie na nich operacji wyszukiwania krawędzi, przeszukiwanie wszerz, przeszukiwanie w głąb oraz sortowanie topologiczne za pomocą algorytmów Kahna i Tarjana. Dodatkowo program umożliwia eksport grafów do formatu TikZ oraz zmianę wybranej reprezentacji grafu jako jedna z komend.

1.2 Prezentacja działania programu

Poniżej przedstawiono przykłady działania programu w trybie **generate** dla wszystkich trzech reprezentacji grafu.

Macierz sąsiedztwa

```
Terminal
$ python3 main.py --generate
type> matrix
nodes> 9
saturation> 33
action> print
  | 1 2 3 4 5 6 7 8 9
-----
1 | 0 1 1 1 1 1 1 1 0
2 | 0 0 0 1 0 0 0 1 0
3 | 0 0 0 0 0 1 0 0 0
4 | 0 0 0 0 0 0 0 0 1
5 | 0 0 0 0 0 0 0 0 0
6 | 0 0 0 0 0 0 0 0 0
7 | 0 0 0 0 0 0 0 0 0
8 | 0 0 0 0 0 0 0 0 1
9 | 0 0 0 0 0 0 0 0 0
action> bfs
1 2 3 4 5 6 7 8 9
action> dfs
1 2 4 9 8 3 6 5 7
action> kahn
1 2 3 5 7 4 8 6 9
action> tarjan
1 7 5 3 6 2 8 4 9
action> exit
Program exited with status: 0
```

Lista następników

```
Terminal
$ python3 main.py --generate
type> list
nodes> 9
saturation> 33
action> print
1 -> 2 -> 6
2 -> 3 -> 6 -> 7 -> 9
3 -> 4 -> 5
4 -> 5
5 -> 7
6
7 -> 8 -> 9
8
9
action> bfs
1 2 6 3 7 9 4 5 8
action> dfs
1 2 3 4 5 7 8 9 6
action> kahn
1 2 3 6 4 5 7 8 9
action> tarjan
1 2 6 3 4 5 7 9 8
action> exit
Program exited with status: 0
```

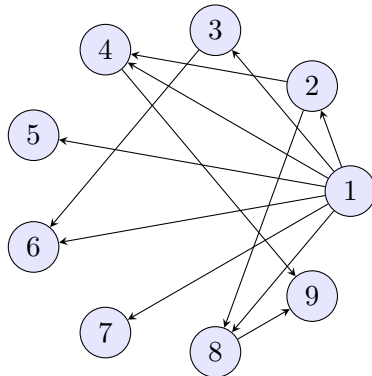
Lista krawędzi

```
Terminal
$ python3 main.py --generate
type> table
nodes> 9
saturation> 33
action> print
(1, 2) (1, 3)
(1, 9) (2, 5)
(2, 7) (2, 9)
(3, 4) (3, 7)
(5, 6) (5, 7)
(6, 8) (7, 9)
action> bfs
1 2 3 9 5 7 4 6 8
action> dfs
1 2 5 6 8 7 9 3 4
action> kahn
1 2 3 5 4 6 7 8 9
action> tarjan
1 3 4 2 5 7 9 6 8
action> exit
Program exited with status: 0
```

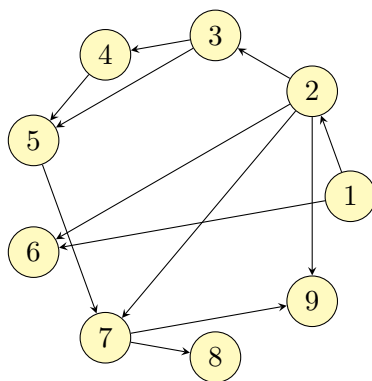
Rysunek 1: Działanie programu.

2 Wizualizacja wygenerowanych grafów

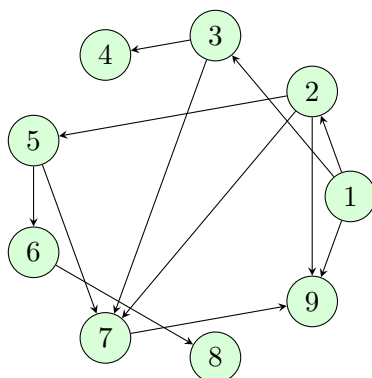
Poniżej znajdują się wizualizacje grafów wygenerowanych w poprzednim punkcie i wyeksportowanych z programu do formatu TikZ.



Rysunek 2: Graf wprowadzony jako macierz sąsiedztwa.



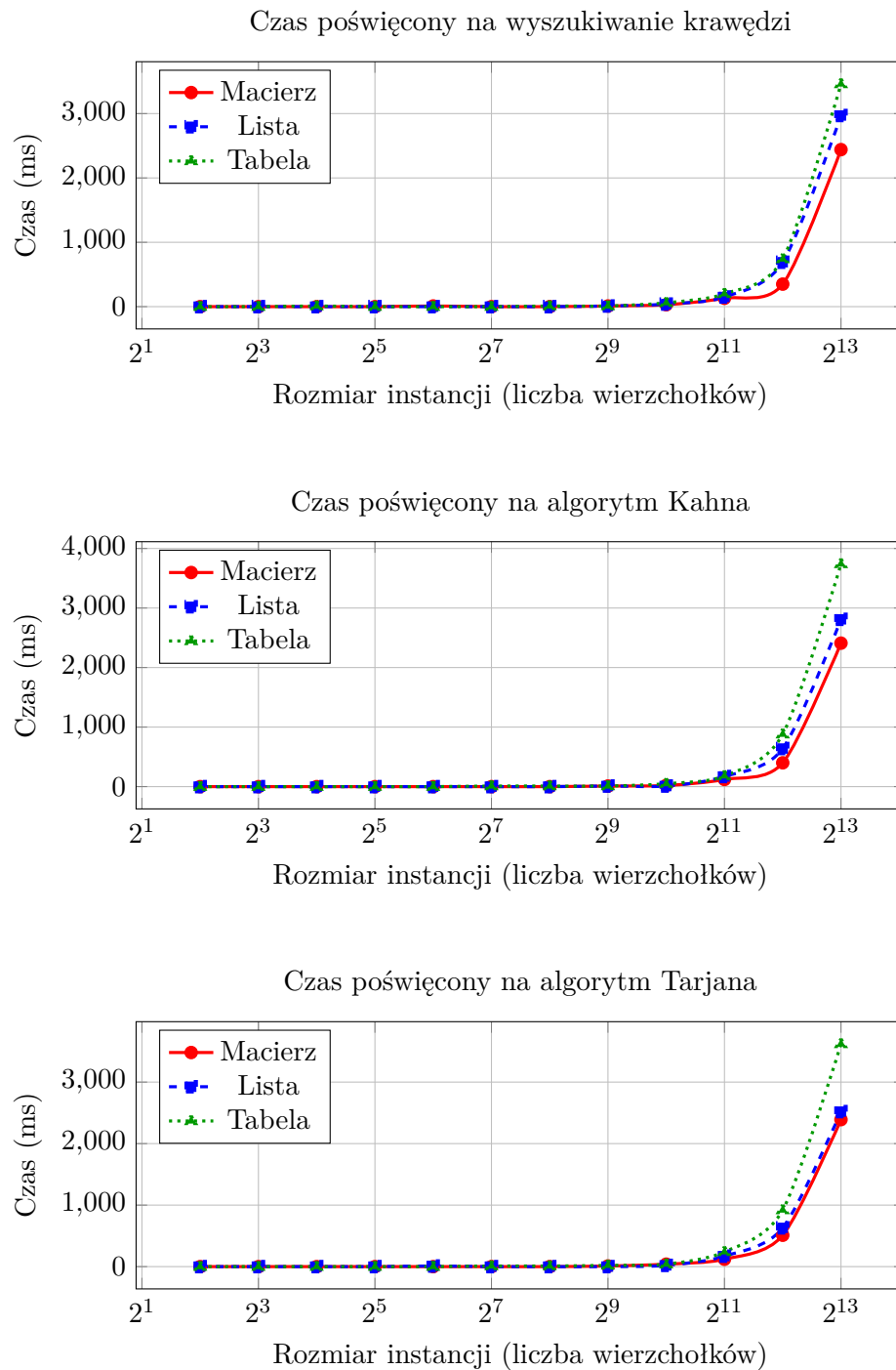
Rysunek 3: Graf wprowadzony jako lista następników.



Rysunek 4: Graf wprowadzony jako lista krawędzi.

3 Wykresy benchmarków

Poniższe wykresy prezentują złożoność obliczeniową trzech operacji programu w zależności od liczby wierzchołków grafu.



Rysunek 5: Zależności czasu obliczeń od rozmiaru instancji grafu.

4 Podsumowanie

W ramach zadania zaimplementowano oraz przeanalizowano wydajność trzech różnych reprezentacji skierowanych grafów acyklicznych: macierzy sąsiedztwa, listy następników oraz listy krawędzi. Wyniki pomiarów jasno wskazują, że wybór struktury danych znacząco wpływa na złożoność poszczególnych operacji. Macierz sąsiedztwa oferuje stałoczasowe sprawdzanie istnienia krawędzi, ale jest bardzo nieoptymalna pamięciowo i obliczeniowo przy przechodzeniu sąsiadów w grafach o małej liczbie krawędzi spośród możliwych. Lista następników okazała się stanowić najlepszy kompromis, zapewniając dobrą wydajność dla algorytmów BFS i DFS (czyli również Kahna i Tarjana). Projekt ten pozwolił na dogłębne zrozumienie zależności między architekturą danych a wydajnością algorytmów sortowania topologicznego w praktyce.